



DFX4ML-ARCH (Progress Update)

Omitted for blind review

CERN, CH-1211 Geneva, Switzerland

Keywords: FPGA, Partial Reconfiguration, Dynamic Function Exchange, PYNQ, Xilinx

Summary

DFX4ML-ARCH presents an FPGA architecture designed to enable partial reconfiguration for machine learning workloads. The project incorporates a self-reconfiguration mechanism, allowing the FPGA fabric to partially reprogram itself. It targets AMD FPGAs equipped with the ICAP3 interface and leverages the PYNQ framework for system integration.

Disclaimer: The technical content, architecture designs, implementation decisions, and all intellectual contributions in this report are entirely the work of the authors. Generative AI tools were used solely for grammar correction and readability enhancement of the written text. No AI-generated technical content, code, or design decisions are included.

Template: This document uses the CERN ATS Note L^AT_EX template, available at <https://www.overleaf.com/latex/templates/cern-ats-note/yzjmgkrvtsqg>.

Contents

1	Introduction	3
1.1	Parameters Under Consideration	3
1.2	Goal	3
2	Architecture Upgrade	4
2.1	DFX Streamer	4
2.1.1	Problems	4
2.1.2	DFX Stream Next Gen	5
2.2	DFX Manager Next Gen	5
2.3	DFX Decoupler Controller Next Gen	6

3	Temporal HLS4ML Vitis Unified Backend Modification	7
3.1	Interface Configuration	7
4	Experiment	8
4.1	Dynamic Region Size vs Reconfigure Time	8
4.2	DFX Streamer Packing Exploration	10
4.2.1	Dynamic Region Layout	10
4.2.2	HLS4ML model	11
4.2.3	Synthesis & implementation	12
4.2.4	Query Explore: Reconfiguration vs Execution Time	12
4.3	Multi-Region (IN-PROGRESS)	14

1 Introduction

The goal of this experiment session is to gather empirical data on the key parameters that govern design trade-offs in an FPGA partial reconfiguration system for machine learning workloads. The results will inform architectural decisions going forward.

1.1 Parameters Under Consideration

Data Batch Size Varies with static region size and model I/O type. A larger batch size is desirable, but it competes directly with the desire for a large static region.

FPGA Static Region Size Should be as small as possible to leave more area for the dynamic region.

FPGA Dynamic Region Size Should be as large as possible to accommodate larger ML models. However, a larger dynamic region increases reconfiguration time.

Reconfiguration Time Should be as small as possible, which pushes toward a smaller dynamic region — directly opposing the goal above.

Model Size Larger ML models are preferred, but they require a larger dynamic region, which increases reconfiguration time.

1.2 Goal

Quantify the actual trends between these parameters so that architectural decisions can be grounded in evidence rather than assumption. Three experiment sessions are proposed:

1. **Region Size vs. Reconfigure Time** — The reconfiguration time depends on the size of the dynamic region (**not the size of the model within the dynamic region**). The experiment aims to explore the trend in reconfiguration time across varying dynamic region sizes.
2. **DFX Streamer Packing Exploration** — We propose an optimised approach to expand the dynamic region as much as possible while retaining as much BRAM/URAM in the static region for the DFX Streamer as possible.
3. **Latency Hiding Technique (IN PROGRESS)** — We propose a technique to hide latency by using the double dynamic region method to keep the system executing continuously.

With data from all sessions, the aim is to identify the operating point that maximises performance, or at minimum to establish a principled basis for deciding the next phase of the design.

2 Architecture Upgrade

2.1 DFX Streamer

2.1.1 Problems

We encountered 2 problems with Vivado and the KV260:

1. **Vivado cannot handle a single large BRAM/URAM array.** Vivado synthesis rejects the `mainMem` variable in `dfx_streamer.v` (line 82) with the following error:

```
[Synth 8-4556] size of variable 'mainMem' is too large to handle;  
the size of the variable is 20,480,000, the limit is 1,000,000.
```

The variable `mainMem` is declared as a flat Verilog register array whose total bit-width (20 480 000) exceeds Vivado's synthesis limit of 1 000 000 bits. Vivado cannot map such a monolithic array to physical BRAM/URAM primitives directly. The fix is to split `mainMem` into multiple smaller arrays, each within the 1 000 000-bit limit, so that Vivado can infer and place individual BRAM/URAM tiles.

2. **Border Latency Problem** The old version of DFX Streamer (Magic Streamer) requires one cycle to save data to BRAM/URAM. The timing constraint is not met. The assumption is that placement restrictions across different Pblocks are responsible. Fig. 1 illustrates the SRC signal (green block), which is constrained to the dynamic region, while the target BRAM spans another region. Vivado cannot perform fan-out optimisation across regions, especially the dynamic region. Moreover, Vivado cannot retime the path either.

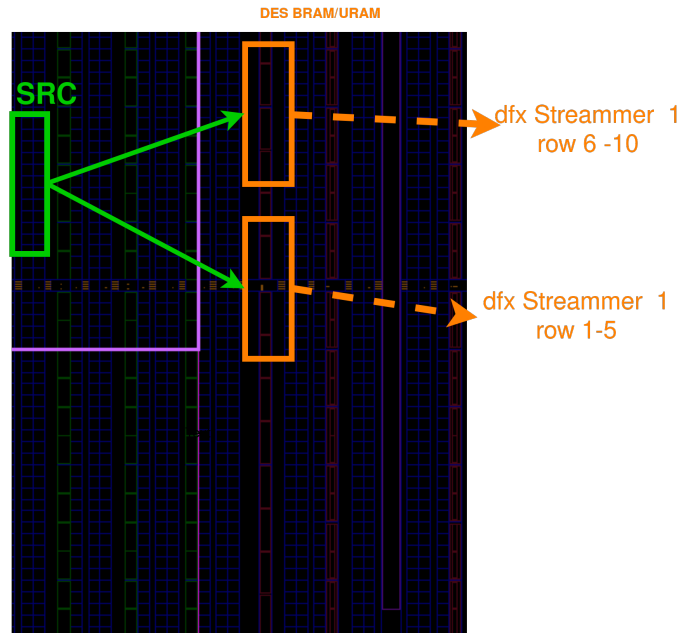


Figure 1: Vivado synthesis error caused by oversized `mainMem` array in `dfx_streamer.v`.

2.1.2 DFX Stream Next Gen

We address the DFX Streamer’s problems by fully re-architecting the memory management system.

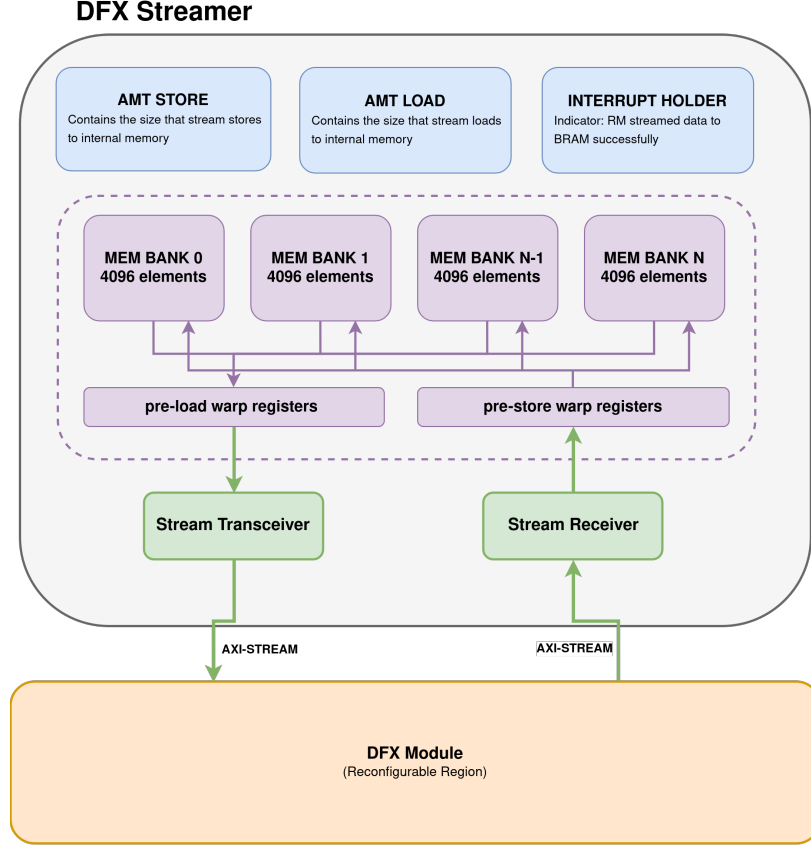


Figure 2: DFX Streamer next-generation architecture.

First, to address the Vivado error, we divide the DFX Streamer’s memory into multiple banks. Each bank has I/O equal to the I/O of the ML model’s front wrap. However, a wrap width that is a multiple of 64 bits is recommended since the URAM’s bit-width is 64 bits.

Moreover, the DFX Streamer packs 4096 entries per bank to maximise resource utilisation for each URAM. If entries are not packed, Vivado cannot decide to merge two memory banks into a single URAM/BRAM, even when they are not accessed simultaneously.

Second, to address the latency problem, we pipeline the URAM/BRAM data transfer (both load and store). On the store side, we add an additional register equal to the wrap size to buffer data coming from the dynamic region. On the load side, we add an additional register equal to wrap size $\times$ number of memory banks; the pipeline supports back-pressure in case the ML model is not ready to receive data from the memory bank.

2.2 DFX Manager Next Gen

In the current experiment session, we used the ML model from HLS4ML’s Vitis Unified Backend; therefore we need AXI-Lite to control the model. We have added two registers to

BANK 0 of the `s_axi_write` module:

- Address `DFX_MAN_ADDR + 0x240` — `prCtrlAddr`: stores the base address of the PR control IP (AXI-Lite slave of the HLS4ML model).
- Address `DFX_MAN_ADDR + 0x280` — `batchSize`: stores the batch size used for a single inference invocation.

Both registers are 32-bit wide (`GLOB_ADDR_WIDTH / GLOB_DATA_WIDTH`) and are decoded from address bits [13:6] in the BANK 0 region (`write_addr[15:14] == 2'b00`).

To accurately collect the execution time for each ML model, we added a 32-bit register to Bank 1 (`0x4024 + 0x64 * RM_ID`) to record the number of elapsed cycles in the system.

2.3 DFX Decoupler Controller Next Gen

For debugging purposes, in the old version the DFX Controller IP owned the DFX decoupler, making it impossible for the PS to bypass control in some situations. We therefore redesigned the ownership control of the DFX decoupler, located at address `0xA001_0000`. The register bit-field definition is shown in Table 1.

Table 1: DFX Decoupler Control Register (`0xA001_0000`) bit-field definition.

Bit	Access	Value	Description
0	WO	1	Ownership of the DFX decoupler belongs to the DFX Controller IP .
		0	Ownership of the DFX decoupler belongs to the PS (Processing System).
1	WO	1	Decouple 0 is coupled (active). Valid only when Bit 0 = 0 (PS ownership).
		0	Decouple 0 is decoupled . Valid only when Bit 0 = 0 (PS ownership).

3 Temporal HLS4ML Vitis Unified Backend Modification

To explore multiple designs that fit into DFX4ML and the KV260, we decided to modify the HLS4ML Vitis Unified backend to enable its interface to connect to the DFX4ML system. However, we did not want to build a full new backend at this stage. We wanted the minimal modifications as follows:

1. The modified backend intermediates the layer’s wrap as bare-metal bits (without converting back to float32 or another type).
2. We use the software prediction result from the old Vitis Unified Backend for result checking.
3. We use the FIFO-depth optimisation from the old Vitis Unified Backend and manually copy it into the new chopped model.

the modified backend is at <https://github.com/Tanawin1701d/hls4ml/tree/VitisUnifiedPartialExp>

3.1 Interface Configuration

The modified backend is invoked via the following call:

```
hls_model = hls4ml.converters.convert_from_keras_model(  
    keras_model,  
    hls_config=cfg,  
    output_dir=output_dir,  
    input_flat=input_flat,  
    output_flat=output_flat,  
    package_as_xo=full_run,  
    project_name=_PROJECT_NAMES[key],  
    **_HLS_PARAMS,  
)
```

Three parameters are specific to the modified backend:

input_flat A boolean flag that controls whether the model’s input wrap is flattened into a 1-D stream of bare-metal bits before being passed to the HLS kernel. When **True**, the input is flattened; when **False**, the system converts from float32 to the target tensor type.

output_flat The output-side counterpart of **input_flat**.

package_as_xo When **True** (bound to **full_run**), the backend runs the full new Vitis HLS flow and packages the resulting kernel as an **.xo** object file ready for linking. When **False**, the system follows the Vitis HLS IP flow for Vivado.

The in detail implementation of the modified interface is at https://github.com/Tanawin1701d/hls4ml/blob/VitisUnifiedPartialExp/hls4ml/templates/vitis_unified/myproject_axi_stream.cpp

4 Experiment

4.1 Dynamic Region Size vs Reconfigure Time

Two build projects were synthesised on the KV260 board (100 MHz system clock) with different dynamic-region sizes. Table 2 summarises the raw measurements.

Table 2: Measured reconfiguration time for two dynamic-region configurations on KV260 (100 MHz).

Project	Size (KB)	Recon. cycles	Recon. time (ms)	Throughput (MB/s)
build_prj_0	1436	367 778	3.678	381.4
build_prj_1	2897	741 674	7.417	381.5

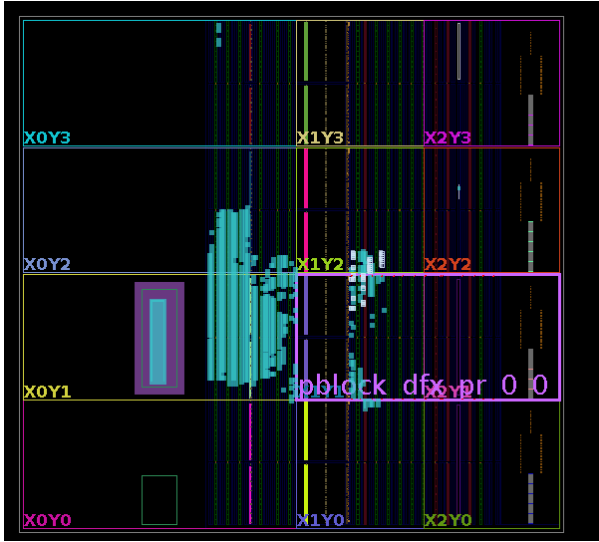


Figure 3: build_prj_0 dynamic region.

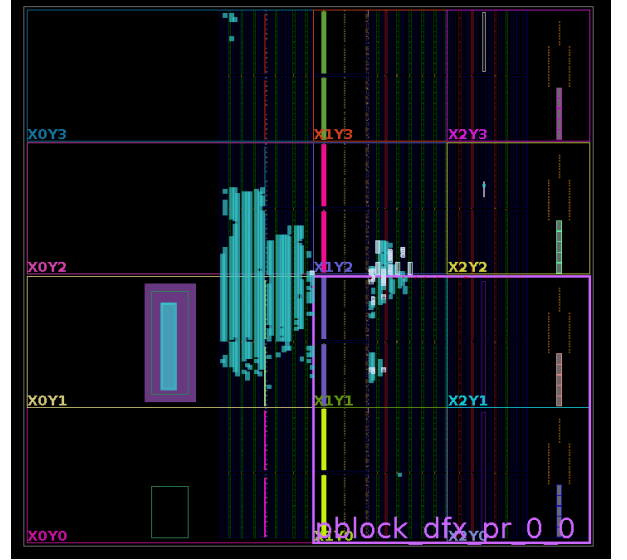


Figure 4: build_prj_1 dynamic region.

The reconfiguration time is linear in bitstream size (ICAP3 throughput is constant at ≈ 381 MB/s), which yields the fitted model:

$$t_{\text{recon}}(\text{KB}) = 0.002560 \times \text{file_size}_{\text{KB}} + 0.0010 \quad [\text{ms}]$$

Figure 5 shows the per-project reconfiguration time and Figure 6 shows the linear relationship with predictions extrapolated up to a 22 MB bitstream.

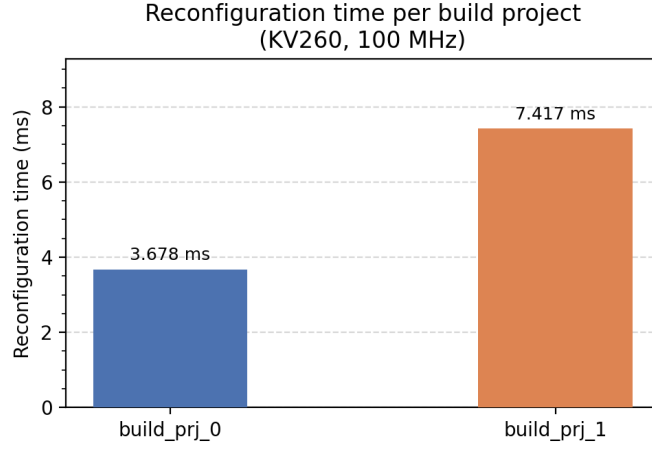


Figure 5: Reconfiguration time per build project on KV260 (100 MHz).

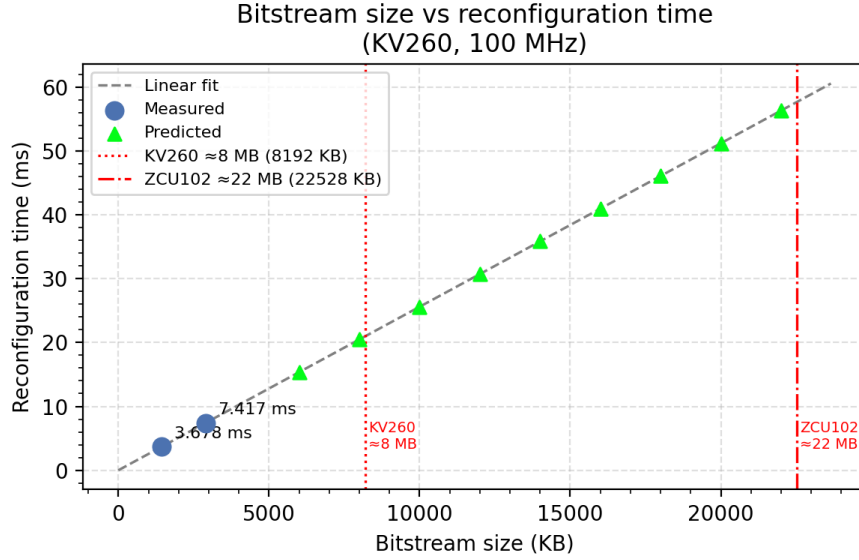


Figure 6: Bitstream file size vs. reconfiguration time with linear fit and predictions up to 22 MB.

Key Takeaway: As previously observed in experiments on the ZCU102, the reconfiguration rate is limited to ~ 399 MB/s; the bottleneck is the ICAP3 port, which accepts only 32 bits per cycle (4 bytes). At 100 MHz, the throughput therefore saturates at 400 MB/s (4 bytes $\times$ 100 MHz).

In addition, the bitstream file size and reconfiguration time do not depend on the number of used logic cells in the FPGA; they depend only on the region size in the fabric.

4.2 DFX Streamer Packing Exploration

4.2.1 Dynamic Region Layout

We allocate the fabric’s resources between the dynamic region and the static region as follows.

1. Static Region: we need it as small as possible since it is not directly involved in ML workloads. However, we require as much on-chip memory as possible, since the DFX Streamer uses it to hold intermediate data.
2. Dynamic Region: we need it as large as possible since it is primarily used for ML workloads. However, a larger region may encroach on the static region’s URAM/BRAM and increase reconfiguration time.

As a result, we design the region arrangement shown in Fig. 7. We compact the static region into the right side of the fabric, which has abundant URAM/BRAM resources. Table 3 summarises the on-chip memory allocated to the static region.

Table 3: Static region on-chip memory resources (KV260).

Type	Blocks	Rows / block	Bit-width	Total (MB)
URAM288	64	4096	64	2.000
BRAM36	96	1024	32	0.375
Total	—	—	—	2.375

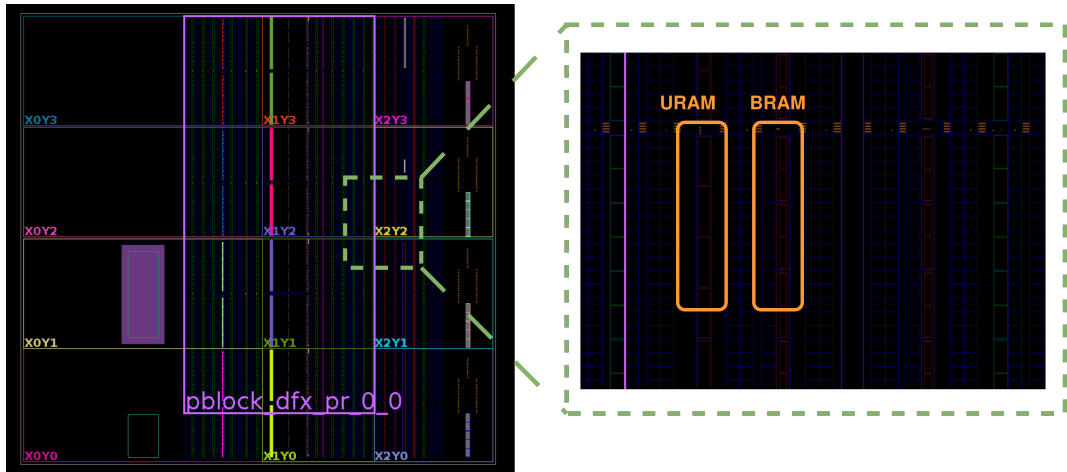


Figure 7: DFX Streamer packing layout for the dynamic region.

4.2.2 HLS4ML model

We used the modified HLS4ML Vitis Unified Backend with the following ML model (image generated by Claude).

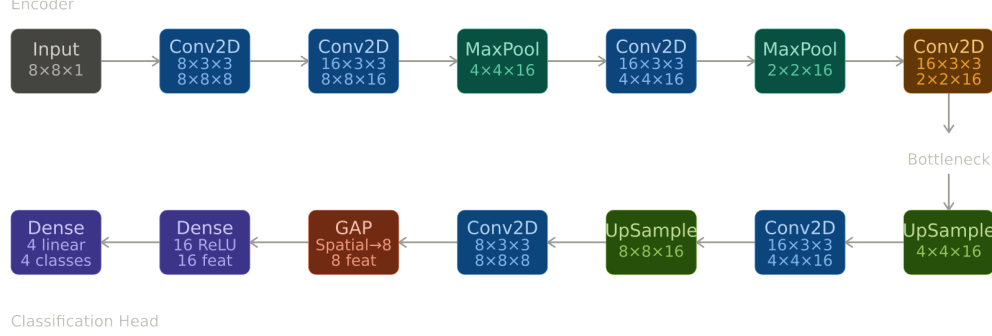


Figure 8: ML model architecture used in the experiment.

The model generated code and debugger are at https://github.com/Tanawin1701d/hls4ml/blob/VitisUnifiedPartialExp/simple_conv_nn_vitis_unified_exp.py

The model was chopped at the bottleneck layer and the FIFO-depth optimisation was copied from the Vitis Unified Backend’s model. The reuse factor was globally set to 8, although the internal HLS4ML logic may adjust it to match kernel alignment.

In addition, we have modified the convolution to fix the BRAM over-utilisation problem for weight storage (<https://github.com/Tanawin1701d/hls4ml/commit/f21523ba25b67e429cc411d2b75378817aeb2cf4>).

In this model, the intermediate layer contains $(2 \times 2 \times 16)$ values at `ap_fixed<16>`. The wavefront width of DFX Streamer (Magic Streamer) is therefore $16 \times 16 = 256$ bits, and each query requires 4 wavefronts (2×2) , giving a **query width of $4 \times 256 = 1024$ bits**.

The number of queries that can be packed into the available memory resources is calculated as follows:

$$\text{Query width} = 2 \times 2 \times 16 \times 16 \text{ bits} = 1024 \text{ bits}$$

$$\text{URAM288 queries} = \frac{\text{blocks} \times \text{rows} \times \text{bit-width}}{\text{query width}} = \frac{64 \times 4096 \times 64}{1024} = 16384 \text{ queries}$$

$$\text{BRAM36 queries} = \frac{\text{blocks} \times \text{rows} \times \text{bit-width}}{\text{query width}} = \frac{96 \times 512 \times 64}{1024} = 3072 \text{ queries}$$

Therefore, we can pack approximately $16384 + 3072 = 19456 \approx 20000$ queries into DFX Streamer.

4.2.3 Synthesis & implementation

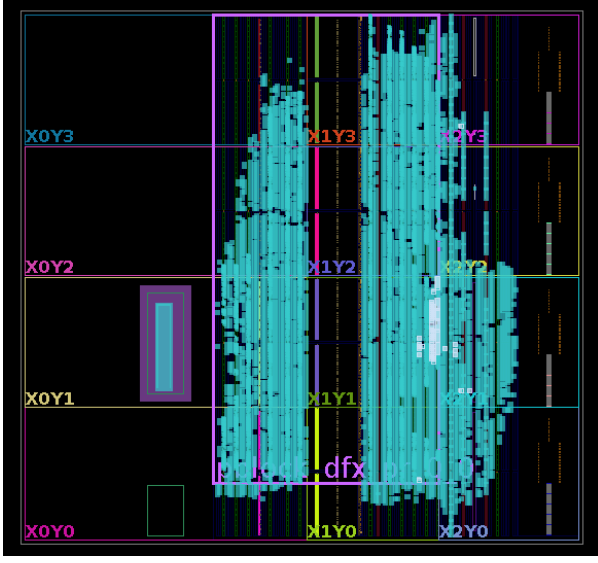


Figure 9: Layout First Half.

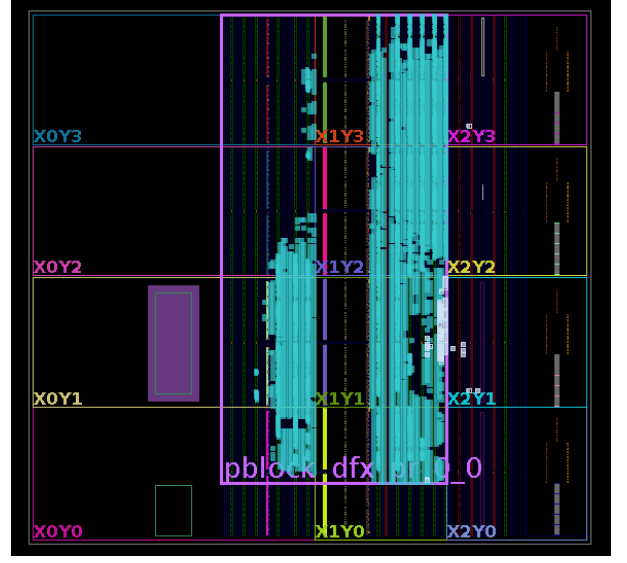


Figure 10: Layout Second Half.

Table 4: Implementation results: timing and hardware resource utilisation (KV260, xck26-sfvc784-2LV-c).

Run	WNS (ns)	LUT	FF	BRAM	URAM	DSP
impl_dfx	0.15	30,964	37,029	59	64	728
child_2_impl_dfx	0.70	27,256	33,511	59	64	480

4.2.4 Query Explore: Reconfiguration vs Execution Time

The reconfiguration time is essentially constant across query counts (it depends only on bitstream size), while the execution time grows linearly with the number of queries. Each configuration is shown at full range alongside a zoomed view (< 5000 queries).

Configuration 1

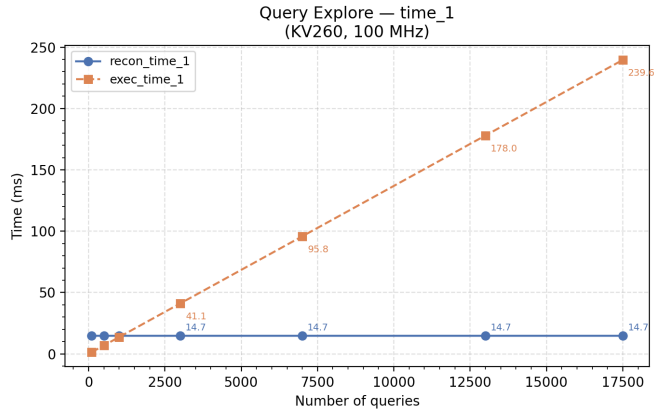


Figure 11: Configuration 1 — full range (KV260, 100 MHz).

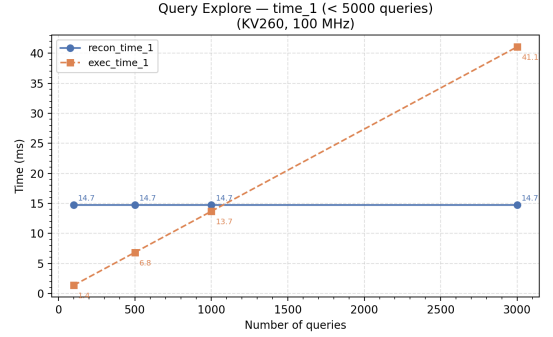


Figure 12: Configuration 1 — zoomed (< 5000 queries).

Configuration 2

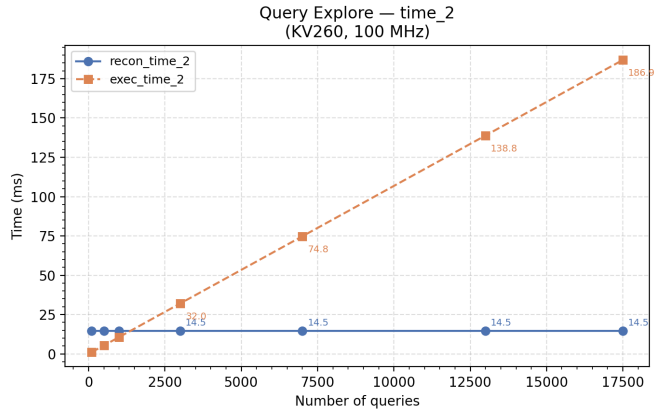


Figure 13: Configuration 2 — full range (KV260, 100 MHz).

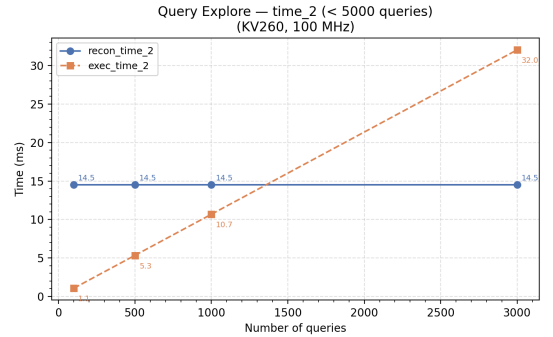


Figure 14: Configuration 2 — zoomed (< 5000 queries).

4.3 Multi-Region (IN-PROGRESS)

Before modifying the entire system to support multiple DFX regions, we would like to conduct a small experiment to understand how to work with multi-region configurations. The aspects we are considering include:

1. DFX Ctrl IP:
 - (a) Explore how to manage signals for multi-region configurations.
 - (b) Explore increasing the ICAP3 clock to 200 MHz (the maximum recommended clock rate for ICAP3 on Zynq UltraScale+) to raise the transfer rate from ~ 400 MB/s to ~ 800 MB/s, while keeping the rest of the fabric at 100 MHz (ML and static region).
2. DFX Manager IP (Magic Sequencer):
 - (a) We are considering upgrading bank 1 to support the latency hiding technique, but the architecture has not yet been finalised.
 - (b) The DFX Manager is currently large and difficult to maintain in Verilog; we may consider rewriting it in SystemVerilog. (I think Claude can do this perfectly.)
3. Tcl Scripts (Block Design):
 - (a) We need to upgrade the Tcl scripts to enable multi-region connections and DFX Streamer arrangement.